

Hunting with Splunk (Lab 3)

LAB 17.3

Scenario

The IT Security manager has provided you with a Splunk instance that has ingested a dataset containing numerous Active Directory attacks. He tasked you with creating Splunk searches that can be used to proactively hunt for them.

Full credit goes to Lukáš Kotlaba (<https://www.linkedin.com/in/lukas-kotlaba/>) for sharing this Splunk dataset as well as the detection methodology this lab covers. Both the dataset and the detection methodology were created as part of Lukáš' Bachelor thesis "[*Detection of Active Directory attacks*](#)"

Goals

The learning objective of this lab is the rule generation ability and becoming more familiar with Splunk.

What you will learn

The learning objective of this lab is to not only get familiar with Splunk's architecture and detection capabilities but also to learn effective Splunk search writing.

Specifically, you will learn how to use Splunk's capabilities in order to:

- Have better visibility over a network
- Respond to incidents timely and effectively
- Proactively hunt for threats

Recommended tools

- Splunk
- Use Firefox browser to connect to Splunk's web interface (<http://demo.ine.local:8000>)

Task 2. Hunt for Kerberoasting

For this task you will have to identify the events/artifacts/traces related to Kerberoasting activity inside an Active Directory environment. Specifically, hunt for:

- **S02D01 - Possible Kerberoasting activity**
 - Look at events 4769 auditing service ticket requests. Filter for any ticket requests with encryption type constants equal to the values of vulnerable cipher suites
- **S02D02 - Excessive service ticket requests from one source**
 - Focus on service accounts that were created for specific services
- **S02D03 - Suspicious external service ticket requests**
 - Examine the *IpPort* and *IpAddress* fields in events 4769. Port values under 1024 and any non-private IP addresses are those of interest
- **S02D04 - Detecting Kerberoasting with a honeypot**
 - Filter all events auditing service requests (4769) for *ServiceName* equal to the honeypot service account (Honeypot01)
- **S02D05 - Detecting Kerberoasting via PowerShell**
 - Focus on the *PowerShell Operational* log. Looks for the occurrence of any service account in the raw data of the PowerShell events. Suppose that the list of service accounts is provided by a lookup file *service_accounts.csv*
- **S03D07 - Possible dumping via DC synchronization**
 - Focus on event 4662 (*An operation was performed on an object*)

Task 3. Hunt for Credential Dumping

For this task you will have to identify the events/artifacts/traces related to Credential Dumping activity inside an Active Directory environment. Specifically, hunt for:

- **S03D01 - Possible dump of lsass.exe (Sysmon events)**

- Focus on Sysmon events with event codes 10 (ProcessAccess) and 8 (CreateRemoteThread)
- **S03D02 - Possible dump of lsass.exe (Windows events)**
 - Use event 4656 (A handle to an object was requested). This event indicates that specific access was requested for an object [19].
- **S03D03 - Creation of a dump file**
 - Sysmon event 11 allows monitoring creation of files together with the process that created them
- **S03D04 - Installation of an unsigned driver**
 - Look for Sysmon event 6 (*Driver loaded*) with the value of field *Signed* equal to *false*
- **S03D05 - Access to GPP honeypot in SYSVOL**
 - Windows event 5145 (*A network share object was checked to see whether client can be granted desired access*) will provide the desired information
- **S03D06 - Possible credential database dumping**
 - Focus on Windows event 4688 and Sysmon event 1 related to binaries that can access *NTDS.dit*

Tasks

Before proceeding to this lab's tasks make sure that you have studied "[Detection of Active Directory attacks](#)", especially if you are not aware of how Active Directory attacks work.

Task 1. Hunt for brute force attacks inside Active Directory

For this task you will have to identify the events/artifacts/traces related to various brute force attacks inside an Active Directory environment. Specifically, hunt for:

- **S01D01 - Logon attempts using a non-existing account (Kerberos)**
 - Look for events 4768 auditing Kerberos authentication ticket (TGT) requests and filters them on the *Status* field equal to code *0x6*. This value means that the submitted username does not exist [3].
- **S01D02 - Logon attempts using a non-existing account (NTLM)**
 - Look for events 4776 auditing NTLM authentication and filters them on the *Status* field equal to code *0xC0000064*. This value means that the submitted username does not exist [4].
- **S01D03 - Excessive failed password attempts from one source (Kerberos)**
 - Filter events 4771 that are logged for several status codes [3]. Among them, there is the code *0x18* which means invalid pre-authentication information, usually a wrong password. The rest of the search is very similar to previous searches, see S01D01 or S01D02 for reference.
- **S01D04 - Excessive failed password attempts from one source (NTLM)**
 - Filter events 4776 that audit NTLM authentication. Status code *0xC000006A* means a logon with misspelled or wrong password [4]. The rest of the search is very similar to previous searches, see S01D01, S01D02 or S01D03 for reference.
- **S01D05 - Excessive failed password attempts towards one account**
 - Filter both Kerberos and NTLM authentication events for status code meaning bad password. That is the status code *0xC000006A* for event 4776 and the status code **0x18* for event 4771
- **S01D06 - Multiple locked accounts from one source**
 - Look into events 4740 - A user account was locked out [5].
- **S01D08 - Logon attempts towards disabled accounts (Kerberos)**
 - Focus on events 4768 with the status code *0x12*, which means that the account is disabled [3]

Task 2. Hunt for Kerberoasting

For this task you will have to identify the events/artifacts/traces related to Kerberoasting activity inside an Active Directory environment. Specifically, hunt for:

- **S02D01 - Possible Kerberoasting activity**
 - Look at events 4769 auditing service ticket requests. Filter for any ticket requests with encryption type constants equal to the values of vulnerable cipher suites
- **S02D02 - Excessive service ticket requests from one source**
 - Focus on service accounts that were created for specific services
- **S02D03 - Suspicious external service ticket requests**
 - Examine the *IpPort* and *IpAddress* fields in events 4769. Port values under 1024 and any non-private IP addresses are those of interest
- **S02D04 - Detecting Kerberoasting with a honeypot**
 - Filter all events auditing service requests (4769) for *ServiceName* equal to the honeypot service account (Honeypot01)
- **S02D05 - Detecting Kerberoasting via PowerShell**
 - Focus on the *PowerShell Operational* log. Looks for the occurrence of any service account in the raw data of the PowerShell events. Suppose that the list of service accounts is provided by a lookup file *service_accounts.csv*
- **S03D07 - Possible dumping via DC synchronization**
 - Focus on event 4662 (*An operation was performed on an object*)

Task 3. Hunt for Credential Dumping

For this task you will have to identify the events/artifacts/traces related to Credential Dumping activity inside an Active Directory environment. Specifically, hunt for:

- **S03D01 - Possible dump of lsass.exe (Sysmon events)**
 - Focus on Sysmon events with event codes 10 (ProcessAccess) and 8 (CreateRemoteThread)

- **S03D02 - Possible dump of lsass.exe (Windows events)**
 - Use event 4656 (A handle to an object was requested). This event indicates that specific access was requested for an object [19].
- **S03D03 - Creation of a dump file**
 - Sysmon event 11 allows monitoring creation of files together with the process that created them
- **S03D04 - Installation of an unsigned driver**
 - Look for Sysmon event 6 (*Driver loaded*) with the value of field *Signed* equal to *false*
- **S03D05 - Access to GPP honeypot in SYSVOL**
 - Windows event 5145 (*A network share object was checked to see whether client can be granted desired access*) will provide the desired information
- **S03D06 - Possible credential database dumping**
 - Focus on Windows event 4688 and Sysmon event 1 related to binaries that can access *NTDS.dit*

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Hunt for brute force attacks inside Active Directory

- **S01D01 - Logon attempts using a non-existing account (Kerberos)**

```
index="ad_hunting" sourcetype=XmlWinEventLog EventCode=4768 Status=0x6 | transaction IPAddress maxpause=5m
maxevents=-1 | where eventcount > 5 | eval Source=if(IPAddress=="::1", Computer, IPAddress) | eval
accounts=mvcount(TargetUserName) | where accounts > 2 | table _time, host, Source, TargetUserName, accounts,
eventcount | sort - _time | rename _time AS "Time", host AS "Host", Source AS "Source Computer",
TargetUserName AS "Target Username", accounts AS "Number of Accounts", eventcount AS "Total Attempts" |
convert ctime(Time)
```

Events	Patterns	Statistics (4)	Visualization		
20 Per Page ▾  Format Preview ▾					
Time ▴ ▾ 	Host ▴ ▾ 	Source Computer ▴ ▾ 	Target Username ▴ ▾ 	Number of Accounts ▴ ▾ 	Total Attempts ▴ ▾ 
01/21/2019 05:49:58.824187	DC01	DC01.test.local	User User1 User12	3	6
01/21/2019 04:03:37.303964	DC01	192.168.56.103	Test01 Test01@test.01 Test01@test.local	3	6
12/18/2018 06:40:05.426691	DC01	192.168.56.103	alkdfjlak gjdfkgj kfdjskf lkaklkoik lkdfjalsk poirwpe	6	12
12/18/2018 06:30:07.919192	DC01	DC01.test.local	adfjnksd asdfasd dsfasdf dskfjlasd qwerwes utyu	6	6

This search looks for events 4768 auditing Kerberos authentication ticket (TGT) requests and filters them on the *Status* field equal to code *0x6*. This value means that the submitted username does not exist [3]. The transaction command is used to group subsequent attempts coming from the same IP address with a maximal pause of 5 minutes between them. Only transactions containing more than five events are preserved. These are the conditions that define parameters of a brute force attack. If there is an IP address of *localhost* in the event, the computer name is taken as a source. The number of different usernames is calculated with the *mvcount* function of the *eval* command. The rule triggers only in the case there are attempts towards more than two different accounts.

How to Implement: Logging on domain controllers needs to be set up to log TGT requests. It can be done by configuring *Audit Kerberos Authentication Service* auditing setting [3]. The constants in the conditions that appear in the search, such as *eventcount*, *maxpause*, and *accounts*, need to reflect *Account Lockout Policy* settings of the domain environment.

Known False Positives: Common false positives are attempts with misspelled usernames. These can be limited by adjusting the search constants precisely for the particular environment.

- S01D02 - Logon attempts using a non-existing account (NTLM)**

```
index="ad_hunting" source=XmlWinEventLog:Security EventCode=4776 Status=0xC0000064 | transaction Workstation
maxpause=5m maxevents=-1 | where eventcount > 5 | eval accounts=mvcount(TargetUserName) | where accounts > 2 |
table _time, host, Workstation, TargetUserName, accounts, eventcount | sort - _time | rename _time AS "Time",
host AS "Host", Workstation AS "Source", TargetUserName AS "Target Username", accounts AS "Number of Accounts",
eventcount AS "Total Attempts"| convert ctime(Time)
```

Events

Patterns

Statistics (4)

Visualization

20 Per Page

Format

Preview

Time	Host	Source	Target Username	Number of Accounts	Total Attempts
01/21/2019 06:24:31.287038	DC01	WINDOWS10	Test1 Test2 Test3 Test4 Test5 Test6	6	6
01/21/2019 06:18:37.490913	DC01	WINDOWS10	Test1 Test2 Test3 Test4 Test5 Test6	6	6
01/21/2019 06:00:12.554032	DC01	WINDOWS10	Test1 Test2 Test3 Test4 Test5 Test6	6	6
01/21/2019 05:53:13.515483	DC01	DC01	Test01 Test02 Test03 Test04 Test05 Test06	6	6

This search looks for events 4776 auditing NTLM authentication and filters them on the *Status* field equal to code *0xC0000064*. This value means that the submitted username does not exist [4]. The *transaction* command is used to group subsequent attempts coming from the same source workstation with a maximal pause of 5 minutes between them. Only transactions containing more than five events are preserved. These are the conditions that define parameters of a brute force attack. The number of different usernames is calculated with *mvcount* function of the *eval* command. The rule triggers only in the case there are attempts towards more than two different accounts.

How to Implement: *Credential Validation* auditing policy needs to be enabled on the monitored systems. The constants in the conditions that appear in the search, such as *eventcount*, *maxpause*, and *accounts*, need to reflect *Account Lockout Policy* settings of the domain environment.

Known False Positives: Common false positives are attempts with misspelled usernames. These can be limited by adjusting the search constants precisely for the particular environment.

- S01D03 - Excessive failed password attempts from one source (Kerberos)**

```
index="ad_hunting" source=XmlWinEventLog:Security EventCode=4771 Status=0x18 | transaction IPAddress
maxpause=5m maxevents=-1 | where eventcount > 5 | eval Source=if(IPAddress=="::1", Computer, IPAddress) | eval
accounts=mvcount(TargetUserName) | table _time, host, Source, TargetUserName, accounts, eventcount | sort -
_time | rename _time AS "Time", host AS "Host", Source AS "Source Computer", TargetUserName AS "Target
Username", accounts AS "Number of Accounts", eventcount AS "Total Attempts" | convert ctime(Time)
```

Events	Patterns	Statistics (9)	Visualization		
20 Per Page ▾  Format Preview ▾					
Time ▴ ▾ 	Host ▴ ▾ 	Source Computer ▴ ▾ 	Target Username ▴ ▾ 	Number of Accounts ▴ ▾ 	Total Attempts ▴ ▾ 
03/11/2019 03:30:36.260659	DC01	192.168.56.103	WINDOWS10\$	1	20
02/17/2019 13:21:55.937982	DC01	192.168.56.106	Administrator	1	40
02/17/2019 13:08:36.237707	DC01	192.168.56.106	Administrator	1	12
02/17/2019 08:20:09.865008	DC01	192.168.56.106	Administrator Database01	2	10

This search detects password brute force attempts coming from one source. Those may be targeting one or multiple accounts, which means this search is also able to identify a password spraying activity. It searches for attempts where Kerberos authentication was used. The rule can be useful even if the lockout policy is applied in the domain because it can detect attempts from adversaries that are aware of the policy. The search filters events 4771 that are logged for several status codes [3]. Among them, there is the code 0x18 which means invalid pre-authentication information, usually a wrong password.

How to Implement: Logging on domain controllers needs to be set up to log TGT requests. It can be done by configuring *Audit Kerberos Authentication Service* auditing setting [3]. The constants in the conditions that appear in the search, such as *eventcount*, *maxpause*, and *accounts*, need to reflect *Account Lockout Policy* settings of the domain environment.

Known False Positives: If there are lots of false positive detections, optionally a condition *where accounts > 1* can be added to the search. However, the condition should be added only in the case that the *Account Lockout Policy* is set up in the environment. If there are multiple IP addresses in use for the same host, the rule can trigger more times for events which should be grouped together in a single result. Users using multiple accounts on one computer may trigger this rule by accidentally typing the wrong password. This behavior can be limited by setting the search constants precisely for the needs of a particular environment.

- **S01D04 - Excessive failed password attempts from one source (NTLM)**

```
index="ad_hunting" source=XmlWinEventLog:Security EventCode=4771 Status=0x18 | transaction IPAddress
maxpause=5m maxevents=-1 | where eventcount > 5 | eval Source=if(IPAddress=="::1", Computer, IPAddress) | eval
accounts=mvcount(TargetUserName) | table _time, host, Source, TargetUserName, accounts, eventcount | sort -
_time | rename _time AS "Time", host AS "Host", Source AS "Source Computer", TargetUserName AS "Target
Username", accounts AS "Number of Accounts", eventcount AS "Total Attempts" | convert ctime(Time)
```

Events	Patterns	Statistics (8)	Visualization								
20 Per Page ▾	✍ Format	Preview ▾									
Time ⬆	✍	Host ⬆	✍	Source ⬆	✍	Target Username ⬆	✍	Number of Accounts ⬆	✍	Total Attempts ⬆	✍
01/21/2019 13:47:36.071633		DC01		WINDOWS10		Win10User		1		10	
01/21/2019 13:39:00.324658		DC01		WINDOWS10		Administrator Win10User Win7User		3		31	
01/21/2019 13:22:18.616172		DC01		WINDOWS10		Win10User		1		10	
01/21/2019 12:56:34.117297		DC01		WINDOWS10		Administrator DefaultAccount Guest Win10User Win7User		8		48	

This search detects password brute force attempts coming from one source. Those may be targeting one or multiple accounts, which means this search is also able to identify a password spraying activity. It searches for attempts where NTLM authentication was used. The rule can be useful even if the lockout policy is applied in the domain because it can detect attempts from adversaries that are aware of the policy. The search filters events 4776 that audit NTLM authentication. Status code 0xC000006A means a logon with misspelled or wrong password [4].

How to Implement: *Credential Validation* auditing policy needs to be enabled on the monitored systems. The constants in the conditions that appear in the search, such as *eventcount*, *maxpause*, and *accounts*, need to reflect *Account Lockout Policy* settings of the domain environment.

Known False Positives: If there are lots of false positive detections, optionally a condition *where accounts > 1* can be added to the search. However, the condition should be added only in the case that the *Account Lockout Policy* is set up in the environment. If there are

multiple IP addresses in use for the same host, the rule can trigger more times for events which should be grouped together in a single result. Users using multiple accounts on one computer may trigger this rule by accidentally typing the wrong password. This behavior can be limited by setting the search constants precisely for the needs of a particular environment.

- **S01D05 - Excessive failed password attempts towards one account**

```
index="ad_hunting" source=XmlWinEventLog:Security ((EventCode=4776 Status=0xC000006A) OR (EventCode=4771 Status=0x18)) | eval src=if(src=="::1", Computer, src) | transaction TargetUserName maxpause=5m maxevents=-1 | eval sources=mvcount(src) | where eventcount > 5 AND sources > 1 | table _time, host, TargetUserName, sources, src, eventcount | sort - _time | rename _time AS "Time", host AS "Host", TargetUserName AS "Target Username", sources AS "Number of Sources", src AS "Source", eventcount AS "Total Attempts" | convert ctime(Time)
```

Events

Patterns

Statistics (3)

Visualization

20 Per Page

Format

Preview

Time	Host	Target Username	Number of Sources	Source	Total Attempts
02/17/2019 13:08:36.237707	DC01	Administrator	2	192.168.56.106 DC01.test.local	14
01/21/2019 12:55:27.352164	DC01	Administrator	2	DC01.test.local WINDOWS10	7
01/21/2019 07:35:54.711095	DC01	Win10User	2	DC01.test.local WINDOWS10	6

This search detects password brute force attempts towards one target account. Unlike previous searches, this rule is set up to capture trials coming from several sources. Both Kerberos and NTLM authentication events are filtered for status code meaning bad password. That is the status code *0xC000006A* for event 4776 and the status code *0x18* for event 4771. Transactions are made on *TargetUserName* field with the *maxpause* parameter set to 5 minutes between single events. Only transactions that contain more than five events in total and events from more than two different sources are preserved as results. These constants should be changed to values that fit the particular environment.

How to Implement: Both *Credential Validation* and *Kerberos Authentication Service* auditing policies need to be enabled on the monitored systems. The constants in the conditions that appear in the search, such as *eventcount*, *maxpause*, and *accounts*, need to reflect *Account Lockout Policy* settings of the domain environment.

Known False Positives: If some hosts use multiple IP addresses, the number of source hosts may be evaluated wrongly. If the condition for sources is *sources > 1*, the rule can trigger false-positively for the same source host, showing as two events. This happens because the event 4776 logs hostname, while event 4771 logs IP addresses only. It can be fixed by using *lookup*. A single user logging on multiple computers in a short time may trigger this rule by accidentally typing the wrong password. This behavior can be limited by setting the search constants precisely.

- **S01D06 - Multiple locked accounts from one source**

```
index="ad_hunting" source=XmlWinEventLog:Security EventCode=4740 | transaction TargetDomainName maxpause=1h maxevents=-1 | eval accounts=mvcount(TargetUserName) | where accounts > 1 | table _time, host, TargetDomainName, TargetUserName, accounts | sort - _time | rename _time AS "Time", host AS "Host", TargetDomainName AS "Source", TargetUserName AS "Target Username", accounts AS "Number of Accounts" | convert ctime(Time)
```

20 Per Page ▾ [Format](#) Preview ▾

Time ▾ Format	Host ▾ Format	Source ▾ Format	Target Username ▾ Format	Number of Accounts ▾ Format
01/21/2019 13:23:47.267625	DC01	WINDOWS10	Administrator Win10User Win7User	3

This search detects several account lockouts made by a single source in a short time. This anomaly can be caused by adversaries trying to guess passwords on multiple accounts in the case they are not aware of the *Account Lockout Policy* in the domain. The search is looking into events 4740 - *A user account was locked out* [5]. The *transaction* command is used with the field *TargetDomainName*. This is the field containing the name of computer account from which logon attempt was received. This applies to XML event format; the field is called *Caller Computer Name* in the standard event format. The condition triggers when there is more than one account locked from the same source computer.

How to Implement: *User Account Management* auditing setting enables logging for account lockout events (4740). The constants in the conditions that appear in the search, such as *maxpause*, and *accounts*, need to reflect *Account Lockout Policy* settings of the domain environment. Replacing the condition *accounts > 1* by *eventcount > 1* returns also repetitive lockout attempts for the same account. It can be useful in the case that accounts are being unlocked automatically after some time.

Known False Positives: A user may lock his own multiple accounts by accidentally typing a wrong password. If some computers are shared among several users, these users can lock their accounts in a short time from a single computer.

- **S01D08 - Logon attempts towards disabled accounts (Kerberos)**

index="ad_hunting" source=XmlWinEventLog:Security EventCode=4768 Status=0x12 | transaction IPAddress maxpause=5m maxevents=-1 | where eventcount > 5 | eval Source=if(IPAddress=="::1", Computer, IPAddress) | eval accounts=mvcount(TargetUserName) | where accounts > 2 | table _time, host, Source, TargetUserName, accounts, eventcount | sort - _time | rename _time AS "Time", host AS "Host", TargetUserName AS "Target Username", accounts AS "Number of Accounts", eventcount AS "Total Attempts" | convert ctime(Time)

EventsPatternsStatistics (0)Visualization

20 Per Page ▼FormatPreview ▼

No results found.

This search identifies repetitive logon attempts with the use of disabled accounts. This rule detects attempts where Kerberos authentication was used. An attacker may want to exploit forgotten accounts that are no longer used in the environment. Or eventually, also default Windows built-in accounts in the case they are disabled. The search is focusing on events 4768 with the status code *0x12*, which means that the account is disabled [3]. Transactions are made on the *IpAddress* field, and only those containing more than five matches are kept in the results. This condition detects signs of a brute force activity. The number of target accounts is evaluated by using the *mvcount* function on the field *TargetUserName*. Only transactions with more than two accounts are displayed.

How to Implement: Logging on domain controllers needs to be configured to audit *Kerberos Authentication Service*. Note that event 4768 with status code *0x12* means a disabled account, whereas the event 4771 with status code *0x12* means that the account is locked.

Known False Positives: Some logon attempts can occur shortly after the account is disabled by an administrator.

Task 2. Hunt for Kerberoasting

- **S02D01 - Possible Kerberoasting activity**

index="ad_hunting" source=XmlWinEventLog:Security EventCode=4769 (TicketEncryptionType=0x1 OR TicketEncryptionType=0x3 OR TicketEncryptionType=0x17 OR TicketEncryptionType=0x18) | eval Source=if(IPAddress=="::1", Computer, IPAddress) | table _time, host, Source, TargetUserName, ServiceName, TicketEncryptionType | sort - _time | rename _time AS "Time", host AS "Host", TargetUserName AS "Target Username", ServiceName AS "Service Name", TicketEncryptionType AS "Ticket Encryption" | convert ctime(Time)

EventsPatternsStatistics (27)Visualization

20 Per Page ▼FormatPreview ▼< Prev12Next >

Time ↕	Host ↕	Source ↕	Target Username ↕	Service Name ↕	Ticket Encryption ↕
04/10/2019 09:47:51.1597332	DC01	192.168.56.104	Win7User@TEST.LOCAL	krbtgt	0x17
04/10/2019 09:37:14.0809748	DC01	192.168.56.105	Win7User@TEST.LOCAL	krbtgt	0x17
04/10/2019 09:35:39.8343732	DC01	192.168.56.105	Win7User@TEST.LOCAL	krbtgt	0x17
03/31/2019 11:19:13.0060172	DC01	DC01.test.local	Administrator@TEST.LOCAL	Honeypot01	0x17
03/31/2019 11:15:31.7738503	DC01	192.168.56.104	Win7User@TEST.LOCAL	Honeypot01	0x17
03/31/2019 11:03:15.1105492	DC01	192.168.56.104	Win7User@TEST.LOCAL	Honeypot01	0x17
03/31/2019 11:03:15.0725865	DC01	192.168.56.104	Win7User@TEST.LOCAL	Database01	0x17

This search finds sources that requested service tickets with weak cipher suites. These encryption types should be no longer used by modern operating systems in the domain. Therefore, they are likely signs of possible Kerberoasting activity. The search looks at events 4769 auditing service ticket requests. It filters for any ticket requests with encryption type constants equal to the values of vulnerable cipher suites. List of all encryption types can be found at [8]. All requests for tickets with these encryption types are displayed.

How to Implement: Domain controllers can log Kerberos TGS service ticket requests (event 4769) by configuring *Audit Kerberos Service Ticket Operations* [8]. As the amount of these events can be quite high, filtering for particular encryption types may also be applied to log forwarding.

Known False Positives: Using older operating systems or services that do not support AES encryption types may create false positives. In such a case, the search needs to be modified to allow the particular encryption type to the specific service.

- **S02D02 - Excessive service ticket requests from one source**

index="ad_hunting" source=XmlWinEventLog:Security EventCode=4769 ServiceName != krbtgt | regex ServiceName != "\\\$" | transaction IPAddress maxpause=5m maxevents=-1 | eval services=mvcount(ServiceName) | where services > 1 | eval Source=if(IPAddress=="::1", Computer, IPAddress) | table _time, host, Source, TargetUserName, services, ServiceName, TicketEncryptionType | sort - _time | rename _time AS "Time", host AS "Host", TargetUserName AS "Target Username", services AS "Number of Services", ServiceName AS "Service Name", TicketEncryptionType AS "Ticket Encryption" | convert ctime(Time)

Events Patterns Statistics (5) Visualization						
20 Per Page Format Preview						
Time	Host	Source	Target Username	Number of Services	Service Name	Ticket Encryption
03/31/2019 11:03:15.072586	DC01	192.168.56.104	Win7User@TEST.LOCAL	2	Database01 HoneyPot01	0x17
03/31/2019 10:58:51.260334	DC01	192.168.56.105	Win7User@TEST.LOCAL	2	Database01 HoneyPot01	0x17
03/31/2019 10:42:20.003056	DC01	192.168.56.105	Administrator@TEST.LOCAL	2	Database01 HoneyPot01	0x17
02/02/2019 03:16:51.240982	DC01	DC01.test.local	Administrator@TEST.LOCAL	2	Database01 HoneyPot01	0x17
02/01/2019 08:12:40.045924	DC01	192.168.56.103	Win10User@TEST.LOCAL	2	Database01 HoneyPot01	0x17

Requests for several different service names (not related to each other) within a short time period from a single account are suspicious. Even more so if weak encryption was used in the service tickets. This search may help to reveal such activities. Service ticket requests for *krbtgt* service and computer account service names (those ending with \$) are filtered out from the results. The search focuses on service accounts that were created for specific services. Subsequent events are grouped on the *IpAddress* field by the transaction command. Number of services in each transaction is calculated to display only results where the number is higher than the one specified in the condition.

How to Implement: Domain controllers can log Kerberos TGS service ticket requests (event 4769) by configuring *Audit Kerberos Service Ticket Operations* [8]. The search constant indicating the number of services needs to be tuned to the needs of a particular domain environment.

Known False Positives: The assessment of whether services are related or not needs to be added to the search if possible. Otherwise it needs to be done manually by a security analyst. Narrowing the search for particular ticket encryption types may prevent false positives. This search may also be combined with detection search S02D01 which could help to prevent false positive detections.

• S02D03 - Suspicious external service ticket requests

```
index="ad_hunting" source=XmlWinEventLog:Security EventCode=4769 IpPort > 0 (IpPort < 1024 OR (NOT (IpAddress=10.0.0.0/8 OR IpAddress=172.16.0.0/12 OR IpAddress=192.168.0.0/16 OR IpAddress=127.0.0.1 OR IpAddress=::1))) | table _time, host, IpAddress, IpPort, TargetUserName, ServiceName, TicketEncryptionType | sort - _time | rename _time AS "Time", host AS "Host", IpAddress AS Source, IpPort AS "Source Port", TargetUserName AS "Target Username", ServiceName AS "Service Name", TicketEncryptionType AS "Ticket Encryption" | convert ctime(Time)
```

20 Per Page Format Preview < Prev 1 2 Next >						
Time	Host	Source	Source Port	Target Username	Service Name	Ticket Encryption
12/18/2018 04:03:45.0819352	DC01	::ffff:192.168.56.103	50154	Win10User@TEST.LOCAL	DC01\$	0x12
12/18/2018 02:47:19.0378882	DC01	::ffff:192.168.56.103	49985	Win10User@TEST.LOCAL	DC01\$	0x12
12/18/2018 02:47:18.9071509	DC01	::ffff:192.168.56.103	49983	Win10User@TEST.LOCAL	krbtgt	0x12

The above is due to the Source being logged improperly. There is no such activity in this lab's dataset.

This search tracks service requests by examining the IP address and port number. Unusual values indicate the use of outbound connection for the service request, which is suspicious. The search examines the *IpPort* and *IpAddress* fields in events 4769. Port values under 1024 and any non-private IP addresses are those of interest. The search displays results whenever such values appear in the request, together with details about the requestor.

How to Implement: Domain controllers can log Kerberos TGS service ticket requests (event 4769) by configuring *Audit Kerberos Service Ticket Operations* [8]. If there is a legitimate service using port beyond 1024 or external IP address, it needs to be whitelisted in the search.

Known False Positives: None identified if whitelisting is done correctly.

• S02D04 - Detecting Kerberoasting with a honeypot

```
index="ad_hunting" source=XmlWinEventLog:Security EventCode=4769 ServiceName=Honeypot01 | eval
Source=if(IpAddress=="::1", Computer, IpAddress) | table _time, host, Source, TargetUserName, ServiceName,
TicketEncryptionType| sort - _time | rename _time AS "Time", host AS "Host", TargetUserName AS "Target
Username", ServiceName AS "Service Name", TicketEncryptionType AS "Ticket Encryption" | convert ctime(Time)
```

Events	Patterns	Statistics (9)	Visualization								
20 Per Page ▾	✍ Format	Preview ▾									
Time ↕	✍	Host ↕	✍	Source ↕	✍	Target Username ↕	✍	Service Name ↕	✍	Ticket Encryption ↕	✍
03/31/2019 11:19:13.0060172		DC01		DC01.test.local		Administrator@TEST.LOCAL		Honeypot01		0x17	
03/31/2019 11:15:31.7738503		DC01		192.168.56.104		Win7User@TEST.LOCAL		Honeypot01		0x17	
03/31/2019 11:03:15.1105492		DC01		192.168.56.104		Win7User@TEST.LOCAL		Honeypot01		0x17	
03/31/2019 10:58:51.2603345		DC01		192.168.56.105		Win7User@TEST.LOCAL		Honeypot01		0x17	

This search uses a detection method for Kerberoasting based on a honeypot. Honeypot is a fake service account that is never really used in the environment, but it is set up to look like a legitimate service account with high privileges assigned. A service ticket requests for this account are only made by an adversary and will be revealed by this search. The search filters all events auditing service requests (4769) for *ServiceName* equal to the honeypot service account (Honeypot01). The search directly produces results that detect malicious TGS requests.

How to Implement: Domain controllers can log Kerberos TGS service ticket requests (event 4769) by configuring *Audit Kerberos Service Ticket Operations* [8]. A fake account (in the search named as Honeypot01) needs to be created, and a fake SPN must be registered for it. Preferably, both account name and SPN should seem as legitimate as possible. The account can also be part of a fake administrator group (set *AdminCount* attribute to 1) to raise its attractiveness for an attacker. More details about the implementation can be found at [9].

Known False Positives: There should not be any false positives. There is no reason to request a service ticket for the honeypot service by a legitimate user.

- **S02D05 - Detecting Kerberoasting via PowerShell**

```
index="ad_hunting" source="WinEventLog:Microsoft-Windows-PowerShell/Operational" (EventCode=4103 OR
EventCode=4104) | transaction Computer maxpause=15m maxevents=-1 | eval raw=_raw | search [| inputlookup
service_accounts.csv | eval raw="*" . account . "*" | fields raw] | where eventcount > 2 | table _time,
Computer, eventcount | sort - _time | rename _time AS "Time", Computer AS "Host", eventcount AS "Number of
Events" | convert ctime(Time)
```

This search is provided for completeness' sake. It won't produce any results.

This search detects attempts to manipulate with service accounts via PowerShell. PowerShell is a tool commonly used by attackers. The search may catch SPN scanning activity or successful acquisition of the service ticket hash. Computers that appear in the results of this search with an extensive amount of events are worth to investigate. The search is built on events from *PowerShell Operational* log. A transaction is created for all subsequent PowerShell events coming from a single workstation. A new field named *raw* is created and assigned the entire raw event. This is a preparation for the full-text search used in the next step. The search looks for the occurrence of any service account in the raw data of the PowerShell events. The list of service accounts is provided by a lookup file *service_accounts.csv*. Only the transactions containing more suspicious events than the specified threshold are displayed in the results.

Task 3. Hunt for Credential Dumping

- **S03D01 - Possible dump of lsass.exe (Sysmon events)**

```
index="ad_hunting" source="xmlwineventlog:microsoft-windows-sysmon/operational" EventCode=8 OR EventCode=10
NOT GrantedAccess=0x1400 NOT GrantedAccess=0x1000 NOT GrantedAccess=0x100000 | where (TargetImage LIKE
"%lsass.exe") |search NOT SourceImage="C:\\Windows\\system32\\wininit.exe" NOT
SourceImage="C:\\Windows\\system32\\csrss.exe" | transaction host, SourceImage, SourceProcessId maxspan=15m |
table _time, host, SourceImage, SourceProcessId, GrantedAccess | sort - _time | rename _time AS "Time", host
AS "Host", SourceImage AS "Process", SourceProcessId AS "Process ID", GrantedAccess AS "Access Mask" | convert
ctime(Time)
```

20 Per Page
Format
Preview

Time	Host	Process	Process ID	Access Mask
07:45:31.945454		0\MsMpEng.exe		0x1410 0x1418 0xffffffff
04/09/2019 13:47:53.797214	DC01	C:\ProgramData\Microsoft\Windows Defender\platform\4.18.1903.4-0\MsMpEng.exe	2168	0x101000 0x1418 0xffffffff
04/08/2019 11:54:17.685507	DC01	C:\ProgramData\Microsoft\Windows Defender\platform\4.18.1903.4-0\MsMpEng.exe	2260	0x101000
04/08/2019 11:43:43.57162	DC01	C:\ProgramData\Microsoft\Windows Defender\platform\4.18.1902.2-0\MsMpEng.exe	2188	0x101000 0xffffffff
04/08/2019 02:47:20.161796	DC01	C:\ProgramData\Microsoft\Windows Defender\platform\4.18.1902.2-0\MsMpEng.exe	2560	0x101000 0x1418 0xffffffff
04/05/2019 06:37:55.691945	DC01	C:\Users\Administrator\Desktop\Sysinternals Suite\calculator.exe	4440	0xffffffff

This search detects possible dump of the LSASS process (*lsass.exe*) memory via Sysmon events. The process memory contains various credentials while the OS is running. To create a dump of the *lsass.exe* process *Administrator* or *SYSTEM* privileges are required, especially *SeDebugPrivilege* or *SeTcbPrivilege* [16]. These are the primary detection artifacts used in this search. It is essential to focus on processes that accessed *lsass.exe* with these privileges. The search is focusing on Sysmon events with event codes 10 (*ProcessAccess*) and 8 (*CreateRemoteThread*). These events are logged when a process creates another process or thread. The search focuses on processes interacting with *lsass.exe* with access mask specifying higher privileges. This is achieved by whitelisting the low-privileged access masks. List of access masks can be found at [17]. It is possible to whitelist some processes that commonly access *lsass.exe*, such as *wininit.exe*. However, it is strongly recommended to specify the full path to the process, as the name and the location of the executable can be easily changed to look like a legitimate process. The transaction command is used to group accesses of the same process in a short period. Process name and ID are displayed for further investigation.

How to Implement: Sysmon must be deployed and configured to log process accesses(event10), and optionally creation of remote threads (event 8). The logs must be ingested and parsed by using TA for Microsoft Sysmon [18]. Whitelisting of processes needs to be done carefully. Only full process paths should be whitelisted. Malicious processes may have their name spoofed to look like a legitimate process.

Known False Positives: Some processes may need to access *lsass.exe* with higher privileges. Depending on the services used in the environment, these processes can be whitelisted to prevent false positives. Examples are common Windows processes or AV solution processes (e.g., Windows Defender).

- S03D02 - Possible dump of lsass.exe (Windows events)**

```
index="ad_hunting" source=XmlWinEventLog:Security EventCode=4656 NOT AccessMask=0x1400 NOT AccessMask=0x1000 NOT AccessMask=0x100000 | where (ObjectName LIKE "%lsass.exe") | search NOT ProcessName="C:\\Windows\\system32\\lsass.exe" | transaction host, ProcessName, ProcessId maxspan=15m | table _time, host, ProcessName, ProcessId, AccessMask | sort - _time | rename _time AS "Time", host AS "Host", ProcessName AS "Process", ProcessId AS "Process ID", AccessMask AS "Access Mask" | convert ctime(Time)
```

20 Per Page ▾FormatPreview ▾< Prev12Next >

Time ▾	Host ▾	Process ▾	Process ID ▾	Access Mask ▾
04/18/2019 00:31:34.739851	DC01	C:\ProgramData\Microsoft\Windows Defender\platform\4.18.1903.4-0\MsMpEng.exe	0x894	0x1410 0x1418 0x1fffff
04/10/2019 09:14:08.072834	DC01	C:\ProgramData\Microsoft\Windows Defender\platform\4.18.1903.4-0\MsMpEng.exe	0x8dc	0x1418 0x1fffff
04/10/2019 07:45:31.945264	DC01	C:\ProgramData\Microsoft\Windows Defender\platform\4.18.1903.4-0\MsMpEng.exe	0x878	0x1410 0x1418 0x1fffff
04/09/2019 13:57:33.893696	DC01	C:\ProgramData\Microsoft\Windows Defender\platform\4.18.1903.4-0\MsMpEng.exe	0x878	0x1418 0x1fffff
04/08/2019 11:54:04.326105	DC01	C:\ProgramData\Microsoft\Windows Defender\platform\4.18.1902.2-0\MsMpEng.exe	0x88c	0x1fffff
04/08/2019 02:57:54.774622	DC01	C:\ProgramData\Microsoft\Windows Defender\platform\4.18.1902.2-0\MsMpEng.exe	0xa00	0x1418 0x1fffff
04/05/2019 06:37:55.691648	DC01	C:\Users\Administrator\Desktop\Sysinternals Suite\calculator.exe	0x1158	0x1fffff

This search detects possible dump of the LSASS process (*lsass.exe*) memory via Windows events. The process memory contains various credentials while the OS is running. To create a dump of the *lsass.exe* process, *Administrator* or *SYSTEM* privileges are needed, especially *SeDebugPrivilege* or *SeTcbPrivilege* [16]. These are the primary detection artifacts used by the search. It is essential to focus on processes that accessed *lsass.exe* with these privileges. The search uses event 4656 (A handle to an object was requested). This event indicates that specific access was requested for an object [19]. Such an object can be the *lsass.exe* process. The same principle for whitelisting low privileged access masks and legitimate processes applies as in S03D01, with the exception that fields are named differently. The transaction command is again used to group accesses of the same process in a short time, and the process name with ID is presented for further investigation.

How to Implement: For event 4656 to be logged, auditing settings under *Object Access* category must be enabled [19]. Whitelisting of processes needs to be done carefully. Only full process paths should be whitelisted. Malicious processes may have their name spoofed to look like a legitimate process.

Known False Positives: Some processes may need to access *lsass.exe* with higher privileges. Depending on the services used in the environment, some processes can be whitelisted to prevent false positives. Examples are common Windows processes or AV solution processes (e.g., Windows Defender).

• S03D03 - Creation of a dump file

```
index="ad_hunting" source="xmlwineventlog:microsoft-windows-sysmon/operational" EventCode=11
TargetFilename=*dmp | table _time, host, Image, ProcessId, TargetFilename | sort - _time | rename _time AS
"Time", host AS "Host", Image AS "Process", ProcessId AS "Process ID", TargetFilename AS "Filename" | convert
ctime(Time)
```

20 Per Page ▾FormatPreview ▾

Time ▾	Host ▾	Process ▾	Process ID ▾	Filename ▾
04/05/2019 06:37:55.6958192	DC01	C:\Users\Administrator\Desktop\Sysinternals Suite\calculator.exe	4440	C:\Users\Administrator\Desktop\Sysinternals Suite\lsass.exe_190405_153755.dmp

A dump file may be created by using many different tools. Even the Task Manager utility integrated into Windows has this capability. Some programs will create a file with *.dmp* extension by default or would not allow changing the filename at all. This search hunts for the creation of such files. Sysmon event 11 allows monitoring creation of files together with the process that created them. It is enough to look for a filename ending with *dmp* and display related information for investigation.

How to Implement: Sysmon must be deployed and configured to log file creation (event 11). The logs must be ingested and parsed by using TA for Microsoft Sysmon [18].

Known False Positives: Dump files created for any process may trigger this rule. They can be created by administrators or developers for debugging purposes. In case this happens regularly, narrowing the search or whitelisting particular files may help to prevent false positives.

• S03D04 - Installation of an unsigned driver

```
index="ad_hunting" source="xmlwineventlog:microsoft-windows-sysmon/operational" EventCode=6 Signed=false |
table _time, host, ImageLoaded, SHA1, SignatureStatus | sort - _time | rename _time AS "Time", host AS "Host",
ImageLoaded AS "Image Loaded", SignatureStatus AS "Signature Status" | convert ctime(Time)
```

Events	Patterns	Statistics (1)	Visualization
20 Per Page ▼ Format Preview ▼			
Time ↕	Host ↕	Image Loaded ↕	SHA1 ↕ Signature Status ↕
02/24/2019 03:06:30.1247007	DC01	C:\Users\Administrator\Desktop\mimikatz_trunk\x64\mimidrv.sys	Expired

Some tools used for credential dumping, such as Mimikatz [16], may attempt to install its own driver to the system. This search aims to detect such attempts by looking at the driver signature. It may also reveal suspicious installations of other drivers to the system beyond the scope of this story. Such events happening on critical systems are surely worth of investigation. The search looks for Sysmon event 6 (*Driver loaded*) with the value of field Signed equal to false. By this, loading of unsigned drivers on the monitored systems can be spotted. Filename with path and hashes can be then used for investigation. Correlation search S03C04 can be used to gain more information.

How to Implement: Sysmon must be deployed and configured to log loading of drivers (event 6). The logs must be ingested and parsed by using TA for Microsoft Sysmon [18].

Known False Positives The number of false positives depends on the usage of different software in the environment. Use of legitimate drivers can be logged and found by this search. If that is the case, whitelisting may be applied to prevent FP detections.

• S03D05 - Access to GPP honeypot in SYSVOL

```
index="ad_hunting" source=XmlWinEventLog:Security EventCode=5145
RelativeTargetName="*test.local\Policies\{12345}*" | transaction IpAddress, SubjectUserSid maxspan=5m
maxevents=-1 | table _time, host, IpAddress, SubjectUserName, SubjectUserSid, SubjectLogonId | sort - _time |
rename _time AS "Time", host AS "Host", IpAddress AS "Source IP", SubjectUserName AS "Username",
SubjectUserSid AS "Account SID", SubjectLogonId AS "Logon ID" | convert ctime(Time)
```

Events	Patterns	Statistics (5)	Visualization
20 Per Page ▼ Format Preview ▼			
Time ↕	Host ↕	Source IP ↕	Username ↕ Account SID ↕ Logon ID ↕
04/05/2019 06:30:24.939395	DC01	fe80::95d7:dffe:e3e9:8025	Administrator S-1-5-21-2678235309-3660057718-2649707153-500 0x3fdb0
04/05/2019 06:25:49.806712	DC01	192.168.56.104	Win7User S-1-5-21-2678235309-3660057718-2649707153-1113 0xa8ac8
04/05/2019 06:24:57.613035	DC01	fe80::95d7:dffe:e3e9:8025	Administrator S-1-5-21-2678235309-3660057718-2649707153-500 0x3fdb0
02/24/2019 06:24:54.195088	DC01	fe80::95d7:dffe:e3e9:8025	Administrator S-1-5-21-2678235309-3660057718-2649707153-500 0x30e5c5
02/24/2019 06:13:28.092941	DC01	fe80::95d7:dffe:e3e9:8025	Administrator S-1-5-21-2678235309-3660057718-2649707153-500 0x30e5c5

Group Policy Preferences can be used to distribute passwords across the domain. GPP data is stored in a domain-wide share *SYSVOL*, to which all *Authenticated Users* have read access. Therefore, *SYSVOL* is a common place for attackers to look for credentials. Such attempts can be detected by creating a honeypot (fake group policy with no effective settings) and logging accesses to it. Any access to this file is suspicious, as there are no reasons to access it. [20] Windows event 5145 (*A network share object was checked to see whether client can be granted desired access*) will provide the desired information. The search filters these events to the honeypot policy file (field *RelativeTargetName*) and displays information about the origin of the action. The *transaction* command is used to group multiple events from the same source in a short time to a single one.

How to Implement: For event 5145 to be logged the *Detailed File Share* auditing setting must be enabled [21].

A fake group policy needs to be created in *SYSVOL* and deny permissions needs to be set for it. For instance: policy *test.local\Policies\{12345}* and permissions *Everyone:Deny*. The policy file structure should look as legitimate as possible; the attacker should not suspect that the policy is a honeypot.

Known False Positives: This rule should not trigger false positives, as there is no reason to access the file except its creation. However, vulnerability scanners may access the share and trigger the rule false-positively.

• S03D06 - Possible credential database dumping

index="ad_hunting" source="xmlwineventlog:security" EventCode=4688 NewProcessName="*vssadmin.exe"

Events (1)PatternsStatisticsVisualization

Format Timeline ▾ — Zoom Out + Zoom to Selection × Deselect1 millisecond per column

List ▾Format20 Per Page ▾

< Hide FieldsAll Fields

SELECTED FIELDS

a host 1

a source 1

a sourcetype 1

INTERESTING FIELDS

a action 1

a app 1

a Caller_Domain 1

a Caller_User_Name 1

a Channel 1

a Computer 1

date_hour 1

date_mday 1

date_minute 1

a date_month 1

date_second 1

a date_wday 1

date_year 1

date_zone 1

a dest 1

i	Time	Event
>	2/24/19 10:08:03.732 AM	<Event xmlns='http://schemas.microsoft.com/win/2004/08/events/event'><System><Provider Name='Microsoft-Windows-Security-Auditing' Guid='{54849625-5478-4994-A5BA-3E3B0328C30D}' /><EventID>4688</EventID><Version>2</Version><Level>0</Level><Task>13312</Task><Opcode>0</Opcode><Keywords>0x8020000000000000</Keywords><TimeCreated SystemTime='2019-02-24T18:08:03.732812000Z' /><EventRecordID>15871493</EventRecordID><Correlation><Execution ProcessID='4' ThreadID='236' /><Channel>Security</Channel><Computer>DC01.test.local</Computer><Security></System><EventData><Data Name='SubjectUserSid'>S-1-5-21-2678235309-3660057718-2649707153-500</Data><Data Name='SubjectUserName'>Administrator</Data><Data Name='SubjectDomainName'>LOCAL</Data><Data Name='SubjectLogonId'>0x30e5c5</Data><Data Name='NewProcessId'>0x8c0</Data><Data Name='NewProcessName'>C:\Windows\System32\vssadmin.exe</Data><Data Name='TokenElevationType'>%1936</Data><Data Name='ProcessId'>0x11dc</Data><Data Name='CommandLine'></Data><Data Name='TargetUserSid'>S-1-0-0</Data><Data Name='TargetUserName'>-</Data><Data Name='TargetDomainName'>-</Data><Data Name='TargetLogonId'>0x0</Data><Data Name='ParentProcessName'>C:\Windows\System32\cmd.exe</Data><Data Name='MandatoryLabel'>S-1-16-12288</Data></EventData></Event> host = DC01 source = XmlWinEventLog:Security sourcetype = XmlWinEventLog

NTDS.dit is the Active Directory database file, an obvious choice for attackers. There are many methods on how to gain access to this file. One of them is to create Volume Shadow Copy of a domain controller. It can be done either by a utility vssadmin.exe [22] or using WMI [23]. WMI can be invoked from PowerShell or by *wmic.exe*. Another method is to used *ntdsutil.exe* - a utility used to build a new domain controller faster [14]. Attackers may also use *reg.exe* to dump hives directly from Registry, where the SAM database is located. The rule detects usage of these methods by looking at command line parameters of newly created processes. The search takes events logging creation of new processes: Windows event 4688 (Sysmon event 1 is better) and filter for known binaries that can access NTDS.dit such as vssadmin.exe.

How to Implement: Sysmon must be deployed and configured to log process creation (event 1). The logs must be ingested and parsed by using TA for Microsoft Sysmon [18]. For event 4688 to be logged, Audit Process Creation setting must be enabled. Furthermore, additional settings must be enabled to include command line parameters in the event [24].

Known False Positives: Volume Shadow Copy Service can be used as a backup solution in the environment. Also, some of the tools included in this search can be used by administrators to perform their operations. In such cases, the search needs to be adjusted to whitelist those activities.

• S03D07 - Possible dumping via DC synchronization

index="ad_hunting" source="xmlwineventlog:security" EventCode=4662 Properties="*1131f6aa-9c07-11d1-f79f-00c04fc2dcd2*" OR Properties="*1131f6ad-9c07-11d1-f79f-00c04fc2dcd2*" OR Properties="*1131f6ab-9c07-11d1-f79f-00c04fc2dcd2*" Caller_User_Name=Administrator

a eventtype 3

a Guid 1

a Handled 1

id 9

a index 1

a Keywords 1

Level 1

linecount 1

a Logon_ID 3

a Name 1

a name 1

a object 1

a ObjectName 1

a ObjectServer 1

a ObjectType 1

Opcode 1

serSid'>S-1-5-21-2678235309-3660057718-2649707153-500</Data><D

Logon_ID

3 Values, 100% of events

Selected

Yes

No

Reports

Top values

Top values by time

Rare values

Events with this field

Values	Count	%
0x107d67	6	50%
0x2a4142	3	25%
0x4e0dc	3	25%

Another method for gaining domain credentials is to abuse the Directory Replication Service Remote protocol used by domain controllers for replication of data. Mimikatz includes a feature DCSync[16],which effectively "impersonates" a domain controller and requests account password data from a real domain controller [14]. This rule may help to hunt it. The search focuses onevent 4662 (*An operation was performed on an object*). This event contains field *Properties*, which enables to determine the access rights used [25]. Therefore, the search filters events based on this field to only those related to DC replication. Then, the Logon ID of the user should be taken from the results and correlated with an authentication event 4624 to see further details about the user session. Most importantly, to determine source workstation and IP address of the activity. The identified IP addresses from the correlation should be those of Domain Controllers, otherwise we are dealing with another endpoint/workstation that has been compromised by an attacker and is impersonating a Domain Controller.

index="ad_hunting" EventCode=4624 0x107d67

Events (2)PatternsStatisticsVisualization

Format TimelineZoom OutZoom to SelectionDeselect1 millisecond per column

ListFormat20 Per Page

	i	Time	Event
< Hide FieldsAll Fields	>	2/28/19 9:00:43.462 AM	<Event xmlns='http://schemas.microsoft.com/win/2004/08/events/event'><System><Provider Name='Microsoft-Windows-Security-Auditing' Guid='{54849625-5478-4994-A5BA-3E3B0328C30D}' /><EventID>4624</EventID><Version>2</Version><Level>0</Level><Task>12544</Task><Opcode>0</Opcode><Keywords>0x8020000000000000</Keywords><TimeCreated SystemTime='2019-02-28T17:00:43.462747700Z' /><EventRecordID>15899531</EventRecordID><Correlation><Execution ProcessID='624' ThreadID='2872' /><Channel>Security</Channel><Computer>DC01.test.local</Computer><Security></System><EventData><Data Name='SubjectUserSid'>S-1-0-0</Data><Data Name='SubjectUserName'></Data><Data Name='SubjectDomainName'></Data><Data Name='SubjectLogonId'>0x0</Data><Data Name='TargetUserSid'>S-1-5-21-2678235309-3660057718-2649707153-500</Data><Data Name='TargetUserName'>Administrator</Data><Data Name='TargetDomainName'>TEST.LOCAL</Data><Data Name='TargetLogonId'>0x107d67</Data><Data Name='LogonType'>3</Data><Data Name='LogonProcessName'>Kerberos</Data><Data Name='AuthenticationPackageName'>Kerberos</Data><Data Name='WorkstationName'></Data><Data Name='LogonGuid'>{0826D947-C4B1-A5C9-6CC6-AC933AC52082}</Data><Data Name='TransmittedServices'></Data><Data Name='LmPackageName'></Data><Data Name='KeyLength'>0</Data><Data Name='ProcessId'>0x0</Data><Data Name='ProcessName'></Data><Data Name='IpAddress'>192.168.56.104</Data><Data Name='IpPort'>49411</Data><Data Name='ImpersonationLevel'>%%1832</Data><Data Name='RestrictedAdminMode'></Data><Data Name='TargetOutboundUserName'>

192.168.56.104 is a Win7 machine (see the accompanying thesis for the lab configuration), certainly not a Domain Controller.

How to Implement: The auditing setting Directory Service Access determines whether the operating system generates audit events when an Active Directory Domain Services (AD DS) object is accessed. This is necessary to log event 4662 [26]. A lookup file containing IP addresses of all domain controllers in the environment must be provided.

Known False Positives: None identified yet.

